

[Technical Guide](@id technical-guide)

This guide explains the implementation details of AlignedAllocs.jl for developers who want to understand how it works or contribute to the project.

Table of Contents

```
Pages = ["technical.md"]
Depth = 3
```

Architecture Overview

AlignedAllocs.jl is structured into three main components:

- 1. **Platform-Specific Allocation** (`AlignedAllocs.jl`)
 - POSIX: Uses `posix_memalign`
 - Windows: Uses `_aligned_malloc` and `_aligned_free`
- 2. **Cache Line Detection** (`precompilation.jl`)
 - Platform-specific system calls
 - Compile-time detection and caching
- 3. **Fixed-Size Array Integration** (`FixedAlignedAllocs.jl`)
 - Wrapper functions for `FixedSizeArrays.jl`
 - Type-stable multi-dimensional allocations

Memory Alignment Fundamentals

What is Memory Alignment?

Memory alignment refers to the arrangement of data in memory at addresses that are multiples of a specific value. For example:

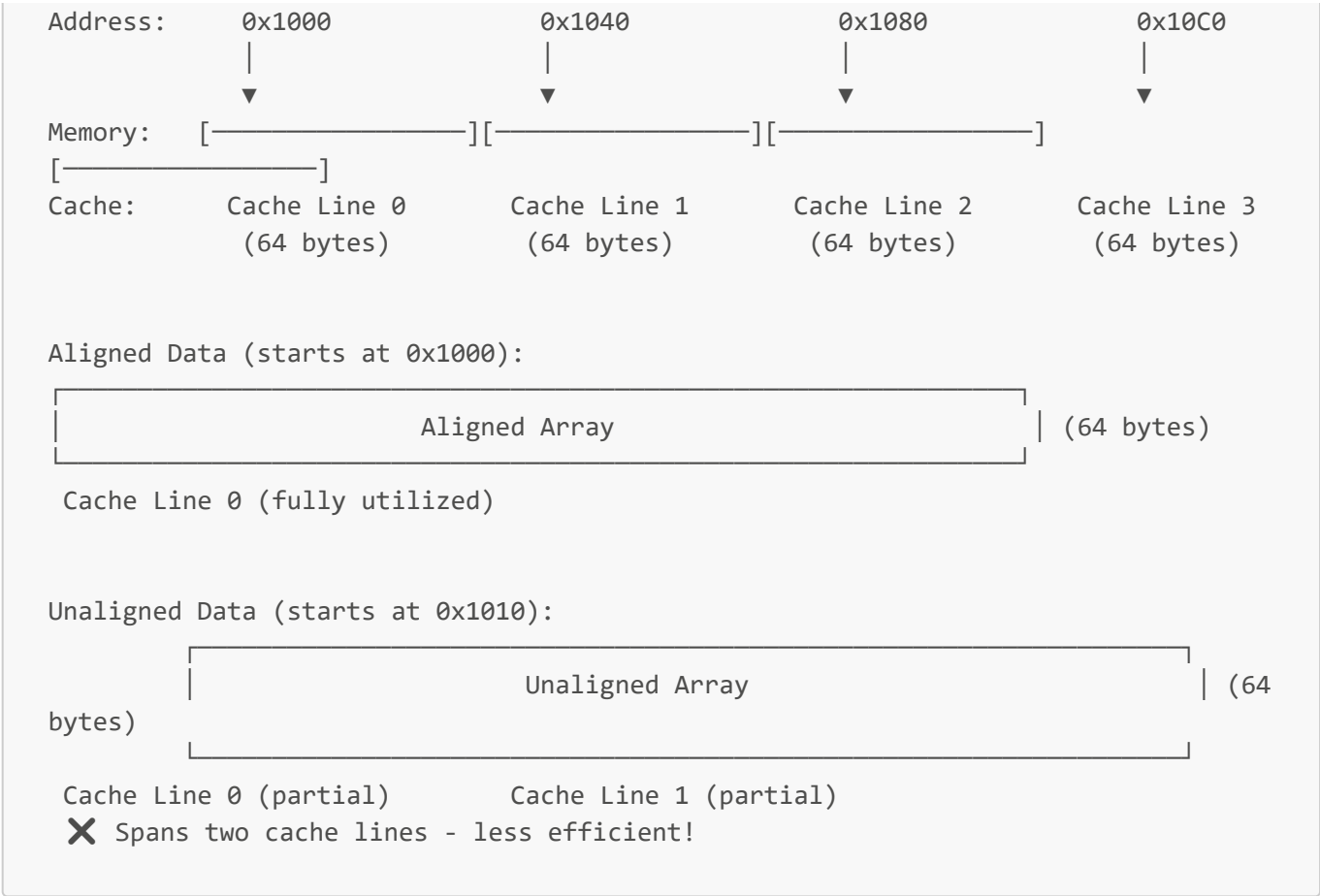
- **16-byte aligned:** Address is a multiple of 16 (e.g., 0x1000, 0x1010, 0x1020)
- **64-byte aligned:** Address is a multiple of 64 (e.g., 0x1000, 0x1040, 0x1080)

Why Alignment Matters

1. CPU Cache Lines

Modern CPUs organize memory into cache lines (typically 64 bytes):

```
Memory Layout (64-byte cache lines):
```



When data is cache-line aligned:

- Each access loads one cache line
- No cache line is split across multiple data structures
- Better spatial locality

When data is not aligned:

- Data may span two cache lines
- Requires loading two cache lines for one access
- Reduced effective cache size

2. SIMD Instructions

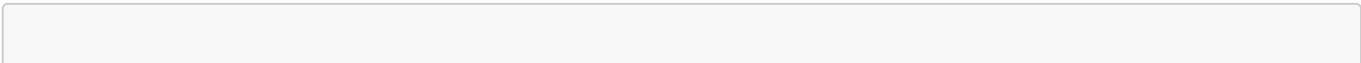
SIMD (Single Instruction, Multiple Data) instructions process multiple values simultaneously:

- **SSE (128-bit)**: Processes 2 doubles or 4 floats at once
- **AVX (256-bit)**: Processes 4 doubles or 8 floats at once
- **AVX-512 (512-bit)**: Processes 8 doubles or 16 floats at once

Some SIMD instructions require or strongly prefer aligned data:

- **Aligned loads**: `movaps`, `vmovapd` (faster)
- **Unaligned loads**: `movups`, `vmovupd` (slower)

Example penalty:



```
# Unaligned: ~10-15% slower for AVX operations
# Aligned: Full SIMD speed
```

3. False Sharing

In multi-threaded code, false sharing occurs when different threads access different variables that share the same cache line:

✗ False Sharing Example:

Thread 1 writes var1 Thread 2 reads var2 Thread 3 writes var3



Cache Line (64 bytes) - SHARED ACROSS THREADS!

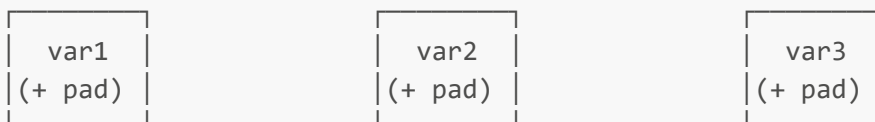
Cache coherency traffic: Thread 1 write $\rightarrow$ invalidates Thread 2's cache

Thread 3 write → invalidates Thread 1's cache

Result: Severe performance degradation (can be 10-100x slower)

☒ Aligned Solution:

```
Thread 1 writes var1      Thread 2 reads var2      Thread 3 writes var3
      ↓                      ↓                      ↓
```



Cache Line 0
(64 bytes)

Cache Line 1
(64 bytes)

Cache Line 2
(64 bytes)

Each variable on separate cache line → No false sharing!

Result: Full parallel performance

Solution: Align each thread's data to separate cache lines:

```
# Bad: false sharing
data = zeros(nthreads())

# Good: each thread has own cache line
data = memalign_vec(Float64, nthreads(); align=64)
```

Platform-Specific Implementation

POSIX Systems (Linux, macOS)

On POSIX-compliant systems, AlignedAllocs uses `posix_memalign`:

```
function memalign_posix(::Type{T}, nitems::Integer, align::Integer) where T
    check_args(T, nitems, align)

    nbytes = Int(_nbytes(T, nitems))
    rawptr = Ref{Ptr{Cvoid}}{C_NULL}

    # Call posix_memalign
    ret = ccall{(:posix_memalign), Cint,
                (Ref{Ptr{Cvoid}}, Csize_t, Csize_t),
                rawptr, Csize_t(align), Csize_t(nbytes))

    ret == 0 || alloc_error(ret)

    ptr = Ptr{T}(rawptr[])
    confirm_alignment(ptr, align)

    # Wrap in Julia Vector with ownership
    vect = unsafe_wrap(Vector{T}, ptr, nitems; own=true)
    return vect
end
```

Key Points:

- `posix_memalign` allocates memory aligned to the specified boundary
- Returns 0 on success, error code otherwise
- Memory is freed automatically by Julia's GC (via `own=true`)

Error Codes:

- `EINVAL (22)`: Invalid alignment (not power of 2 or not multiple of word size)
- `ENOMEM (12)`: Insufficient memory

Windows Systems

On Windows, AlignedAllocs uses `_aligned_malloc` and `_aligned_free`:

```
function memalign_windows(::Type{T}, nitems::Integer, align::Integer) where T
    check_args(T, nitems, align)

    nbytes = Int(_nbytes(T, nitems))

    # Call _aligned_malloc from msvcrt
    ptr = ccall{(:_aligned_malloc, "msvcrt"), Ptr{T},
                (Csize_t, Csize_t), Csize_t(nbytes), Csize_t(align))

    (ptr == C_NULL) && alloc_error(ENOMEM)

    confirm_alignment(ptr, align)
```

```

# Wrap in Julia Vector WITHOUT ownership
vect = unsafe_wrap(Vector{T}, ptr, nitems; own=false)
freed = Ref(false)

# Custom finalizer to call _aligned_free
finalizer(vect) do _
    if !freed[]
        ccall((:_aligned_free, "msvcrt"), Cvoid, (Ptr{Cvoid},), Ptr{Cvoid}
(ptr))
        freed[] = true
    end
end

return vect
end

```

Key Differences from POSIX:

- `_aligned_malloc` returns pointer directly (or NULL on failure)
- Requires explicit `_aligned_free` (cannot use standard `free`)
- Uses custom finalizer instead of Julia's automatic cleanup
- `own=false` prevents Julia from calling wrong deallocation function

Why the Freed Flag?

The `freed` flag prevents double-free:

- Finalizers may run multiple times
- `freed[]` ensures we only call `_aligned_free` once

Cross-Platform Wrapper

The main entry point dispatches to the appropriate implementation:

```

@inline function memalign_vec(::Type{T}, nitems::Integer;
align::Integer=CACHE_LINE_SIZE) where T
    @static Sys.iswindows() ? memalign_windows(T, nitems, align) :
memalign_posix(T, nitems, align)
end

```

The `@static` ensures compile-time dispatch (no runtime overhead).

Cache Line Size Detection

AlignedAllocs automatically detects the CPU's cache line size at compile time.

Detection Strategy

```

const CACHE_LINE_SIZE = let size = try
    _detect_cache_line_size()
catch err
    @warn "Failed to detect cache line size; using fallback."
    FallbackCacheLineSize
end
size > 0 ? size : FallbackCacheLineSize
end

```

Fallback Value: 64 bytes (most common on modern CPUs)

Platform-Specific Detection

macOS (Darwin)

Uses `sysctlbyname` to query hardware properties:

```

line_size = Ref{Cuint}(0)
size_ref = Ref{Csize_t}(sizeof(Cuint))
ret = ccall(:sysctlbyname, Cint,
    (CString, Ptr{Cvoid}, Ptr{Csize_t}, Ptr{Cvoid}, Csize_t),
    "hw.cachelinesize", line_size, size_ref, C_NULL, 0)
return ret == 0 ? Int(line_size[]) : FallbackCacheLineSize

```

Linux and Unix

Uses `sysconf` with `_SC_LEVEL1_DCACHE_LINESIZE`:

```

const _SC_LEVEL1_DCACHE_LINESIZE = 190

line_size = ccall(:sysconf, Clong, (Cint,), _SC_LEVEL1_DCACHE_LINESIZE)
return line_size > 0 ? Int(line_size) : FallbackCacheLineSize

```

Windows

Uses `GetLogicalProcessorInformationEx`:

```

function _windows_cache_line_size()::Int
    bufsize = Ref{UInt32}(0)

    # First call: get required buffer size
    ccall(:(GetLogicalProcessorInformationEx, "kernel32"),
        Cint, (Cint, Ptr{Cvoid}, Ptr{UInt32}),
        RELATION_CACHE, C_NULL, bufsize)

```

```

buffer = Base.Vector{UInt8}(undef, bufsize[])

# Second call: get processor information
ret = ccall{(:GetLogicalProcessorInformationEx, "kernel32"),
            Cint, (Cint, Ptr{UInt8}, Ptr{UInt32}),
            RELATION_CACHE, buffer, bufsize)

# Parse buffer to find L1 cache line size
GC.@preserve buffer begin
    base_ptr = pointer(buffer)
    offset = 0
    limit = Int(bufsize[])

    while offset < limit
        entry_size = Int(unsafe_load{Ptr{UInt32}}(base_ptr + offset + 4))
        level = unsafe_load{Ptr{UInt8}}(base_ptr + offset + 8)

        if level == 1
            line_size = unsafe_load{Ptr{UInt16}}(base_ptr + offset + 10)
            return Int(line_size)
        end

        offset += entry_size
    end
end

return FallbackCacheLineSize
end

```

Binary Structure of CACHE_DESCRIPTOR:

Offset	Size	Field
0	4	RelationshipType
4	4	Size
8	1	Level
9	1	Associativity
10	2	LineSize
...		

Precompilation

Cache line size is detected once during package precompilation:

```

@setup_workload begin
    @compile_workload begin
        _detect_cache_line_size()
    end
end

```

This ensures:

- No runtime overhead for detection
- Consistent behavior across package usage
- Cached result in precompiled system image

Validation and Safety

Argument Validation

Before allocation, arguments are validated:

```
@inline function check_args(::Type{T}, nitems::Integer, align::Integer) where T
    isbitstype(T) || throw(ArgumentError("element_type ($T) must be a
`bitstype`"))
    nitems > 0 || throw(ArgumentError("element_count ($nitems) must be > 0"))
    is_alignment_valid(align) || throw(ArgumentError("Alignment ($align) must be
2^p where p >= 4"))
    return nothing
end
```

Checks:

1. **Type is bitstype:** Only types without pointers (like `Int64`, `Float64`) are allowed
2. **Positive count:** Must allocate at least one element
3. **Valid alignment:** Must be power of 2 and at least 16

Alignment Validation

Validates the alignment requirement:

```
@inline is_alignment_valid(align::Integer) = align >= 16 && ((align - 1) & align)
== 0
```

How it works:

- `align >= 16`: Minimum 16-byte alignment (CPU requirement)
- `(align - 1) & align == 0`: Power of 2 test

Power of 2 Test:

```
64  = 0b01000000
63  = 0b00111111
&   = 0b00000000  ✓ (power of 2)

48  = 0b00110000
```



```
47 = 0b00101111
&  = 0b00100000  X (not power of 2)
```

Overflow Protection

Prevents integer overflow when computing byte size:

```
@generated function _nbytes(::Type{T}, n::Integer) where {T}
    sz = sizeof(T)
    return :(Base.checked_mul(n, $sz))
end
```

Uses `@generated` for compile-time type size calculation and `Base.checked_mul` for runtime overflow detection.

Post-Allocation Verification

After allocation, confirms the memory is actually aligned:

```
@inline function confirm_alignment(ptr::Ptr, align::Integer)
    mask = UInt(align - 1)
    if (UInt(ptr) & mask) != 0
        # Alignment failed - free memory and throw error
        if Sys.iswindows()
            ccall(:_aligned_free, "msvcrt"), Cvoid, (Ptr{Cvoid},), ptr)
        else
            ccall(:free, "Libc"), Cvoid, (Ptr{Cvoid},), ptr)
        end
        errmsg = "aligned memory allocation failed: returned address $(UInt(ptr))
is not aligned to $(align) bytes"
        throw(ErrorException(errmsg))
    end
    return nothing
end
```

How alignment check works:

```
# For 64-byte alignment:
mask = 63 # 0b00111111

# If ptr = 0x1040 (aligned):
0x1040 & 0x003F = 0x0000  ✓

# If ptr = 0x1042 (not aligned):
0x1042 & 0x003F = 0x0002  X
```

Zero Initialization

`memalign_clear_vec` provides zero-initialized memory:

```
@inline function memalign_clear_vec(::Type{T}, nitems::Integer;
align::Integer=CACHE_LINE_SIZE) where T
    vect = memalign(T, nitems; align)

    # Zero the memory
    Base.GC.@preserve vect begin
        nbytes = Int(_nbytes(T, nitems))
        ptr = Base.unsafe_convert{Ptr{UInt8}, Base.pointer(vect)}
        zeromem(ptr, nbytes)
    end

    return vect
end

@inline function zeromem(ptr::Ptr{UInt8}, nbytes::Int)
    Base.memset(ptr, 0x00, nbytes)
    return nothing
end
```

Key Points:

- Uses `Base.memset` for efficient zeroing
- `@preserve` ensures GC doesn't move array during zeroing
- Zeros entire byte range (not element-by-element)

Fixed-Size Array Support

Generic Construction Pattern

All fixed-size functions follow this pattern:

```
@inline function _aligned_construct(::Type{T}, dims::Tuple{Vararg{Integer,N}},
                                   align::Integer, allocator, builder) where
{T,N}
    # 1. Normalize and validate dimensions
    sdims = _normalize_dims(dims)

    # 2. Compute total length with overflow check
    len = _checked_length(sdims)

    # 3. Allocate flat buffer
    buffer = allocator(T, len; align=align)

    # 4. Build final structure
    return builder(buffer, sdims)
end
```

This pattern separates:

1. **Validation** - Check dimensions are valid
2. **Computation** - Calculate total size
3. **Allocation** - Get aligned memory
4. **Construction** - Build final data structure

Dimension Validation

```
@inline function _normalize_dims(dims::Tuple{Vararg{Integer,N}}) where {N}
    N == 0 && throw(ArgumentError("at least one dimension is required"))
    ntuple{Val(N)}() do i
        dim = Int(dims[i])
        dim > 0 || throw(ArgumentError("dimension $i ($dim) must be > 0"))
        dim
    end
end
```

Ensures:

- At least one dimension
- All dimensions positive
- Converts to `Int` for type stability

Overflow-Safe Length Calculation

```
@inline function _checked_length(dims::NTuple{N,Int}) where {N}
    len = Int(1)
    for dim in dims
        len, overflow = Base.Checked.mul_with_overflow(len, dim)
        overflow && throw(OverflowError("dimension product overflowed Int"))
    end
    len
end
```

Uses `mul_with_overflow` to detect when `dim1 * dim2 * ... * dimN` exceeds `typemax{Int}`.

Sequential Aligned Vectors

`memalign_seq` creates multiple vectors, each independently aligned:

```
function memalign_seq(::Type{T}, nvectors, nitems_per_vector;
    align=CACHE_LINE_SIZE) where {T}
    # Compute stride: elements between vector start addresses
    stride = align ÷ sizeof(T)
```

```
# Allocate single buffer with space for all vectors
storage = memalign_clear_fix(T, stride * nvectors; align)

# Create vector views, each at aligned offset
vectors = GC.@preserve storage begin
    baseptr = Base.unsafe_convert{Ptr{T}}, storage)
    ntuple(nvectors) do i
        # Offset to i-th aligned position
        ptr = baseptr + (i-1) * stride * sizeof(T)
        # Wrap without ownership (storage owns memory)
        chunk = Base.unsafe_wrap{Vector{T}}, ptr, nitems_per_vector; own=false)
        # Convert to fixed-size array
        FixedSizeArrays.new_fixed_size_array(chunk, (nitems_per_vector,))
    end
end

vectors
end
```

Memory Layout:

Sequential Aligned Vectors (align=64, nitems=10, nvectors=3):

Vector 1 (10 items) (80 bytes)	Padding (48 bytes)	Vector 2 (10 items) (80 bytes)	Padding	...
-----------------------------------	-----------------------	-----------------------------------	---------	-----

^

Offset 0
2*stride
(aligned to 64)
64)

^

Offset stride
(aligned to 64)

^

Offset
(aligned to

stride = align ÷ sizeof(T) = 64 ÷ 8 = 8 elements per stride
Each vector uses 10 elements, but is allocated stride elements

Actual memory access:
Vector[1]: elements 0-9 (uses 10, allocated 8 stride positions)
Vector[2]: elements 8-17 (uses 10, allocated 8 stride positions)
Vector[3]: elements 16-25 (uses 10, allocated 8 stride positions)

Note: Each vector starts at aligned boundary (0, 64, 128, ...)

Each vector starts at an aligned boundary.

Alignment Detection Algorithm

The alignment function finds the maximum alignment of an array:

```
@inline function alignment(xs::AbstractArray)
    addr = UInt(pointer(xs))
    return addr == 0 ? 0 : Int(addr & -addr)
end
```

How `addr & -addr` works:

```
addr = 0x1040 # Binary: ...0001000001000000
-addr          # Two's complement
addr & -addr   # Isolates rightmost set bit

# Example:
# 0x1040 = 0b...0001000001000000
# -0x1040 = 0b...1110111111000000 (two's complement)
# -----
# & =      0b...0000000001000000 = 64
```

This extracts the largest power of 2 that divides the address.

Tuple Alignment

For tuples of arrays, returns minimum alignment:

```
@inline function alignment(xs::NTuple{N,T}) where {N,T}
    minimum(map(alignment, xs))
end
```

Useful for checking if multiple arrays have compatible alignment.

Performance Considerations

Type Stability

All functions are type-stable (return type depends only on argument types):

```
# Type-stable examples
v = memalign_vec(Float64, 100) # :: Vector{Float64}
v = memalign_clear_vec(Int32, 50) # :: Vector{Int32}
a = alignment(v) # :: Int
```

Type stability enables Julia's compiler optimizations:

- Function inlining
- Devirtualization
- Dead code elimination

- LLVM optimizations

Inlining

Most functions are marked `@inline` for zero-overhead abstractions:

```
@inline function memalign_vec(::Type{T}, nitems::Integer;
    align::Integer=CACHE_LINE_SIZE) where T
    # ...
end
```

Small functions are inlined into call sites, eliminating function call overhead.

Generated Functions

`_nbytes` uses `@generated` to compute size at compile time:

```
@generated function _nbytes(::Type{T}, n::Integer) where {T}
    sz = sizeof(T)
    return :(Base.checked_mul(n, $sz))
end
```

At compile time: `sizeof(T)` is known At runtime: Only multiplication is performed

GC Interaction

POSIX Systems:

```
vect = unsafe_wrap(Vector{T}, ptr, nitems; own=true)
```

- Julia's GC automatically calls `free` on the pointer

Windows Systems:

```
vect = unsafe_wrap(Vector{T}, ptr, nitems; own=false)
finalizer(vect) do _
    ccall((:_aligned_free, "msvcrt"), Cvoid, (Ptr{Cvoid},), ptr)
end
```

- Custom finalizer ensures proper cleanup
- `own=false` prevents Julia from calling wrong free function

Testing Strategy

Property-Based Tests

Test invariants that should always hold:

```
@test alignment(memalign_vec(T, n; align=a)) >= a # Alignment guarantee
@test length(memalign_vec(T, n)) == n             # Size correctness
@test all(iszero, memalign_clear_vec(T, n))        # Zero initialization
```

Edge Cases

Test boundary conditions:

```
memalign_vec(T, 1)           # Minimum size
memalign_vec(T, 1_000_000)   # Large allocation
memalign_vec(T, n; align=16) # Minimum alignment
memalign_vec(T, n; align=4096) # Large alignment
```

Error Conditions

Verify proper error handling:

```
@test_throws ArgumentError memalign_vec(String, 10) # Non-bits type
@test_throws ArgumentError memalign_vec(Int64, 0)    # Zero size
@test_throws ArgumentError memalign_vec(Int64, 10; align=10) # Invalid alignment
```

Platform-Specific Tests

Test on multiple platforms:

- Linux (POSIX)
- macOS (Darwin)
- Windows (MSVC)

Performance Tests

Benchmark critical operations:

```
using BenchmarkTools

@benchmark memalign_vec(Float64, 10000)
@benchmark memalign_clear_vec(Float64, 10000)
@benchmark alignment($v)
```

Contributing Guidelines

Code Style

- Follow Julia standard style guide
- Use meaningful variable names
- Add docstrings for public functions
- Include type annotations where helpful
- Mark internal functions with leading underscore

Adding New Features

1. **Design:** Discuss on GitHub first
2. **Implementation:** Follow existing patterns
3. **Tests:** Add comprehensive tests
4. **Documentation:** Update all docs
5. **Benchmarks:** Compare performance

Platform Support

When adding platform-specific code:

```
@static if Sys.iswindows()
    # Windows implementation
elseif Sys.isapple()
    # macOS implementation
elseif Sys.isunix()
    # Unix/Linux implementation
else
    error("Unsupported platform")
end
```

Safety Checklist

- ☐ Validate all inputs
- ☐ Check for overflow
- ☐ Confirm alignment post-allocation
- ☐ Proper memory cleanup (finalizers on Windows)
- ☐ GC-safe pointer operations
- ☐ Thread-safe if applicable

Future Enhancements

Potential improvements:

1. **NUMA Support:** Allocate on specific NUMA nodes
2. **Huge Pages:** Support for huge page allocations
3. **Memory Pools:** Reusable allocation pools
4. **Custom Allocators:** Plugin architecture for custom allocators
5. **GPU Memory:** Extend to CUDA/ROCm
6. **Memory Mapping:** Support for memory-mapped files

References

- [Intel® 64 and IA-32 Architectures Optimization Reference Manual](#)
 - [POSIX memalign specification](#)
 - [Windows _aligned_malloc documentation](#)
 - [Julia Memory Management](#)
 - [Cache-Oblivious Algorithms](#)
-

Glossary

A

Alignment: The property of a memory address being a multiple of a specific value (typically a power of 2). For example, 64-byte alignment means the address is divisible by 64.

AVX (Advanced Vector Extensions): Intel SIMD instruction set extension supporting 256-bit operations.

AVX-512: Extension of AVX supporting 512-bit operations (8 doubles or 16 floats).

B

Bits Type: A Julia type without pointers, represented directly in memory. Examples: `Int64`, `Float64`, custom structs with only bits-type fields.

C

Cache Line: The minimum unit of data transfer between main memory and CPU cache. Typically 64 bytes on modern CPUs.

Cache Line Size: The size of a cache line, detected automatically by `AlignedAllocs` (usually 64 bytes).

ccall: Julia's mechanism for calling C functions from Julia code.

F

False Sharing: Performance degradation in multi-threaded code when threads access different variables that share the same cache line, causing unnecessary cache coherency traffic.

Finalizer: A function that runs when an object is garbage collected, used to clean up resources (e.g., free memory on Windows).

M

Memory Alignment: See **Alignment**.

memset: C library function for setting memory to a specific byte value, used for zero-initialization.

N

NUMA (Non-Uniform Memory Access): Computer architecture where memory access time depends on memory location relative to processor.

P

POSIX: Portable Operating System Interface, a family of standards for Unix-like operating systems.

posix_memalign: POSIX function for allocating aligned memory.

Power of 2: A number that can be expressed as 2^n (e.g., 16, 32, 64, 128, 256).

S

SIMD (Single Instruction, Multiple Data): Parallel processing technique where one instruction operates on multiple data elements simultaneously.

SSE (Streaming SIMD Extensions): Intel SIMD instruction set supporting 128-bit operations.

Stride: The number of elements between consecutive elements in a particular dimension of an array.

System Page: Fixed-length block of memory (typically 4096 bytes) used by virtual memory systems.

T

Type Stability: Property where a function's return type can be inferred from argument types at compile time, enabling optimization.

Two's Complement: Method of representing signed integers in binary, used in the alignment detection algorithm.

V

Vectorization: Automatic or manual conversion of scalar operations to SIMD operations for parallel processing.

Z

Zero-Initialization: Setting all bytes of allocated memory to zero before use.

Questions about the implementation? Open an issue on GitHub!